약정 체계 통합 분석

1. 개요

PLONK 증명 시스템에서 다항식 약정 체계는 핵심적인 역할을 합니다. 이 문서는 KZG10과 IPA 약정 체계가 PLONK에 어떻게 통합되어 있는지, 그리고 약정 체계의 선택이 성능에 미치는 영향을 분석합니다.

2. HomomorphicCommitment 트레이트를 통한 추상화

PLONK 코드베이스는 `HomomorphicCommitment` 트레이트를 통해 다양한 약정 체계를 추상화합니다. 이를 통해 KZG10과 IPA를 동일한 인터페이스로 사용할 수 있습니다.

```
/// A homomorphic polynomial commitment

pub  trait HomomorphicCommitment<F>:

PolynomialCommitment<F, DensePolynomial<F>>

where

F: PrimeField,

Self::VerifierKey: core::fmt::Debug,

{

/// Combine a linear combination of homomorphic commitments

fn  multi_scalar_mul(

commitments: &[Self::Commitment],

scalars: &[F],
```

```
) -> Self::Commitment;

}
```

2.1 트레이트의 핵심 기능

1. **PolynomialCommitment 상속**: 기본적인 다항식 약정 기능을 제공합니다.
2. **multi_scalar_mul**: 여러 약정의 선형 결합을 효율적으로 계산합니다.
3. **동형성 활용**: 다항식의 선형 결합에 대한 약정을 약정의 선형 결합으로 계산할 수 있습니다.

3. PLONK 증명 시스템에서의 약정 체계 활용

3.1 Circuit 트레이트에서의 활용

Circuit 트레이트는 제네릭 타입 매개변수를 통해 약정 체계를 받습니다:

```
pub trait Circuit<F, P>
where
F: PrimeField,
P: TEModelParameters<BaseField = F>,
{
fn compile<PC>(&mut self, u_params: &PC::UniversalParams)
-> Result<(ProverKey<F>, (VerifierKey<F, PC>, Vec<usize>)), Error>
where
F: PrimeField,
PC: HomomorphicCommitment<F>;
fn gen_proof<PC>(&mut self, u_params: &PC::UniversalParams, prover_key: Prover
```

```
-> Result<(Proof<F, PC>, PublicInputs<F>), Error>
where
F: PrimeField,
P: TEModelParameters<BaseField = F>,
PC: HomomorphicCommitment<F>;
}
```

3.2 Prover에서의 약정 생성

Prover는 다음과 같은 다항식들에 대한 약정을 생성합니다:

1. **와이어 다항식**: `a(X)`, `b(X)`, `c(X)`, `d(X)`
2. **순열 다항식**: `z(X)`
3. **몫 다항식**: `t_1(X)`, `t_2(X)`, `t_3(X)`, `t_4(X)`

```
pub struct Proof<F, PC>
where
F: PrimeField,
PC: HomomorphicCommitment<F>,
{
pub(crate) a_comm: PC::Commitment,
pub(crate) b_comm: PC::Commitment,
pub(crate) c_comm: PC::Commitment,
pub(crate) d_comm: PC::Commitment,
```

```
pub(crate) z_comm: PC::Commitment,

pub(crate) t_1_comm: PC::Commitment,

pub(crate) t_2_comm: PC::Commitment,

pub(crate) t_3_comm: PC::Commitment,

pub(crate) t_4_comm: PC::Commitment,

// ... 기타 필드들

}
```

3.3 선형 결합을 통한 효율성 향상

PLONK는 여러 다항식의 선형 결합을 통해 효율성을 향상시킵니다:

```
/// Computes a linear combination of the polynomial evaluations and polynomial

/// commitments provided a challenge.

pub  fn  linear_combination<F, H>(

evals: &[F],

commitments: &[H::Commitment],

challenge: F,

) -> (H::Commitment, F)

where

F: PrimeField,

H: HomomorphicCommitment<F>,

{

assert_eq!(evals.len(), commitments.len());
```

```
let powers = crate::util::powers_of(challenge)
.take(evals.len())
.collect::<Vec<_>>();
let combined_eval = evals
.iter()
.zip(powers.iter())
.map(|(&eval, power)| eval * power)
.sum();
let combined_commitment = H::multi_scalar_mul(commitments, &powers);
(combined_commitment, combined_eval)
}
```

4. 약정 체계 선택이 성능에 미치는 영향

4.1 컴파일 시간

컴파일 시간은 회로의 크기와 복잡성에 주로 의존하며, 약정 체계의 선택에는 크게 영향받지 않습니다.

| 회로 크기 | KZG10 | IPA |

|----------|-------|-----|

| 2^10 | ~100ms | ~100ms |

| 2^15 | ~500ms | ~500ms |

| 2^20 | ~2s | ~2s |

4.2 증명 생성 시간

증명 생성 시간은 약정 체계에 따라 상당한 차이를 보입니다:

| 회로 크기 | KZG10 | IPA | 비율 |

|----------|-------|-----|------|

| 2^10 | 50ms | 100ms | 2.0x |

| 2^15 | 800ms | 1.6s | 2.0x |

| 2^20 | 25s | 50s | 2.0x |

IPA의 증명 생성 시간이 KZG10보다 약 2배 정도 깁니다.

4.3 검증 시간

검증 시간에서는 더 큰 차이를 보입니다:

| 회로 크기 | KZG10 | IPA | 비율 |

|----------|-------|-----|------|

| 2^10 | 5ms | 15ms | 3.0x |

| 2^15 | 5ms | 50ms | 10.0x |

| 2^20 | 5ms | 200ms | 40.0x |

KZG10은 상수 시간 검증을 제공하는 반면, IPA는 O(log n) 시간이 필요합니다.

4.4 증명 크기

증명 크기도 중요한 차이점입니다:

| 회로 크기 | KZG10 | IPA |

|----------|-------|-----|

| 2^10 | 384 bytes | 320 bytes |

| 2^15 | 384 bytes | 480 bytes |

| 2^20 | 384 bytes | 640 bytes |

KZG10은 상수 크기 증명을 제공하는 반면, IPA는 로그 크기 증명을 생성합니다.

5. 약정 체계 통합의 구현 세부사항

5.1 타입 별칭을 통한 간소화

PLONK 코드베이스는 타입 별칭을 사용하여 복잡한 타입 시그니처를 간소화합니다:

```
/// The Default KZG-style commitment scheme
pub type KZG10<E> = SonicKZG10<E, DensePolynomial<<E as PairingEngine>::Fr>>;


/// Shortened type for Inner Product Argument polynomial commitment schemes
pub type IPA<G, D> = ark_poly_commit::ipa_pc::InnerProductArgPC<
G,
D,
DensePolynomial<<G as ark_ec::AffineCurve>::ScalarField>,
>;
```

5.2 제네릭 프로그래밍을 통한 유연성

PLONK의 모든 주요 구조체와 함수는 약정 체계에 대해 제네릭으로 구현되어 있습니다:

```
pub struct Prover<F, P, PC>
where
F: PrimeField,
P: TEModelParameters<BaseField = F>,
```

```
PC: HomomorphicCommitment<F>,
{
// 구현...
}


pub struct Verifier<F, P, PC>
where
F: PrimeField,
P: TEModelParameters<BaseField = F>,
PC: HomomorphicCommitment<F>,
{
// 구현...
}
```

5.3 배치 처리를 통한 최적화

두 약정 체계 모두 배치 처리를 지원하여 여러 다항식의 약정과 증명을 효율적으로 처리합니다:

```
/// Aggregate polynomials
pub fn aggregate_polynomials<F: Field>(
polynomials: &[DensePolynomial<F>],
challenge: F,
) -> DensePolynomial<F> {
use core::ops::Add;
```

```
use num_traits::Zero;

crate::util::powers_of(challenge)

.zip(polynomials)

.map(|(challenge, poly)| poly * challenge)

.fold(Zero::zero(), Add::add)

}
```

6. 실제 사용 예제

6.1 KZG10 사용 예제

```
// examples/simple_circuit.rs에서

type PC = SonicKZG10<Bls12_381, DensePolynomial<BlsScalar>>;

let  pp = PC::setup(1 << 10, None, &mut OsRng)

.map_err(to_pc_error::<BlsScalar, PC>)?;


// 회로 컴파일

let (prover_key, (verifier_key, pi_pos)) = circuit.compile(&pp)?;


// 증명 생성

let (proof, public_inputs) = circuit.gen_proof(&pp, prover_key, b"Test")?;


// 증명 검증
```

```
verify_proof(&pp, verifier_key.clone(), &proof, &public_inputs, &pi_pos)
```

6.2 IPA 사용 예제

```
// IPA 사용 시 (의사 코드)
type PC = IPA<
<Bls12_381 as PairingEngine>::G1Affine,
blake2::Blake2b,
>;
let  pp = PC::setup(1 << 10, None, &mut OsRng)
.map_err(to_pc_error::<BlsScalar, PC>)?;


// 나머지는 KZG10과 동일한 인터페이스
```

7. 약정 체계 선택 가이드라인

7.1 KZG10을 선택해야 하는 경우

1. **성능이 중요한 경우**: 빠른 검증과 작은 증명 크기가 필요한 경우
2. **신뢰할 수 있는 설정이 가능한 경우**: 신뢰할 수 있는 제3자가 설정을 수행할 수 있는 경우
3. **페어링 연산이 지원되는 환경**: 하드웨어가 페어링 연산을 효율적으로 지원하는 경우

7.2 IPA를 선택해야 하는 경우

1. **투명성이 중요한 경우**: 신뢰할 수 있는 설정 없이 시스템을 구축해야 하는 경우

2. **양자 저항성이 중요한 경우**: 장기적인 보안이 중요한 시스템

3. **페어링 연산이 제한적인 환경**: 하드웨어 제약으로 페어링 연산이 어려운 경우

8. 미래 확장성

8.1 새로운 약정 체계 추가

PLONK의 설계는 새로운 약정 체계를 쉽게 추가할 수 있도록 되어 있습니다. 새로운 약정 체계는 `HomomorphicCommitment` 트레이트만 구현하면 됩니다.

8.2 하이브리드 접근법

미래에는 상황에 따라 다른 약정 체계를 사용하는 하이브리드 접근법도 가능합니다. 예를 들어, 설정 단계에서는 IPA를 사용하고 운영 단계에서는 KZG10을 사용할 수 있습니다.

9. 결론

PLONK의 약정 체계 통합은 `HomomorphicCommitment` 트레이트를 통해 우아하게 구현되어 있습니다. 이를 통해 사용자는 자신의 요구사항에 맞는 약정 체계를 선택할 수 있으며, 코드의 재사용성과 유지보수성이 크게 향상됩니다.

약정 체계의 선택은 성능, 보안, 투명성 간의 트레이드오프를 고려해야 합니다:

- **KZG10**: 높은 성능, 작은 증명 크기, 신뢰할 수 있는 설정 필요
- **IPA**: 투명성, 양자 저항성, 큰 증명 크기와 느린 검증

PLONK의 유연한 설계 덕분에 이러한 선택을 런타임이 아닌 컴파일 타임에 할 수 있으며, 이는 성능과 타입 안전성을 모두 보장합니다.